Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Source Code: `https://github.com/atishmandal/CS3807-Deep-Learning-Lab/tree/main/lab7`

1. Objective

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

`https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist`

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$\boxed{x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}}$$

where x is the input image and $\hat{x}$ is its reconstruction.

Execution details

- **Data:** the first 10,000 training and first 2,000 test images of Keras MNIST, scaled to $[0, 1]$. The last 10% of the training subset (1,000 images) was held out for validation, so the models were fitted on 9,000 images; the test images were never used during training.
- **FC-AE, CAE and denoising CAEs:** Adam (learning rate 10^{-3}), batch size 128, 20 epochs, binary cross-entropy. The latent layer of the FC-AE is linear (no ReLU).
- **Denoising CAEs:** two separate models with the CAE architecture, one trained on Gaussian noise ($\sigma = 0.2$) and one on salt-and-pepper noise ($p = 0.10$); each was evaluated at all three levels of its own noise type.
- **VAE:** latent dimension 2, 30 epochs, batch size 128. Encoder: Conv(32, stride 2)–Conv(64, stride 2)–Dense(16)–$\mu, \log \sigma^2$. Decoder: Dense(3136)–ConvTranspose(64)–ConvTranspose(32)–ConvTranspose(1, sigmoid). Reconstruction (BCE) and KL losses are summed per image and averaged over the batch; reconstruction metrics use $z = \mu$.
- **Metrics:** MSE and MAE over all test pixels; SSIM computed per image (data range 1) and averaged. Random seed 42.

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

```
Original image
Reconstructed image
Reconstruction error
```

For the denoising model, display:

```
Original clean image
Noisy image
Denoised reconstruction
```

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.
The encoder maps the input to a lower-dimensional representation:

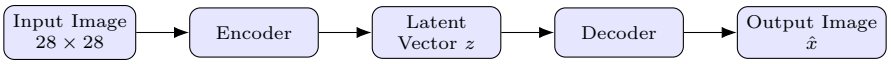
$$z = f_\theta(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_\phi(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^{N} \|x_i - \hat{x}_i\|^2.$$



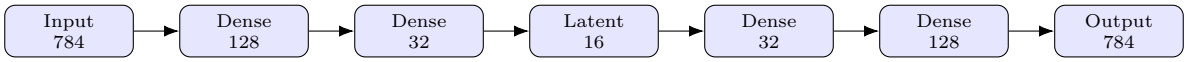
The latent vector is a compressed representation of the input.

7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.
Use the following architecture:



Use ReLU activation in hidden layers and sigmoid activation in the output layer.
Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

8. Required Analysis of the Basic Autoencoder

Original versus Reconstructed Images

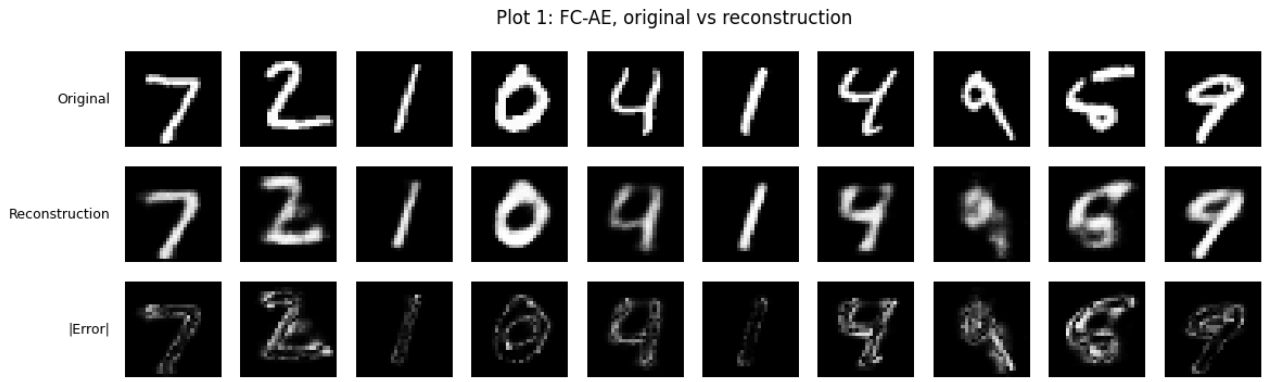
Display at least 10 test images in a grid.

For every image, show:

Original | Reconstruction.

Students must comment on:

- preservation of digit shape;
- loss of fine details;
- blurred regions;
- differences between easy and difficult samples.

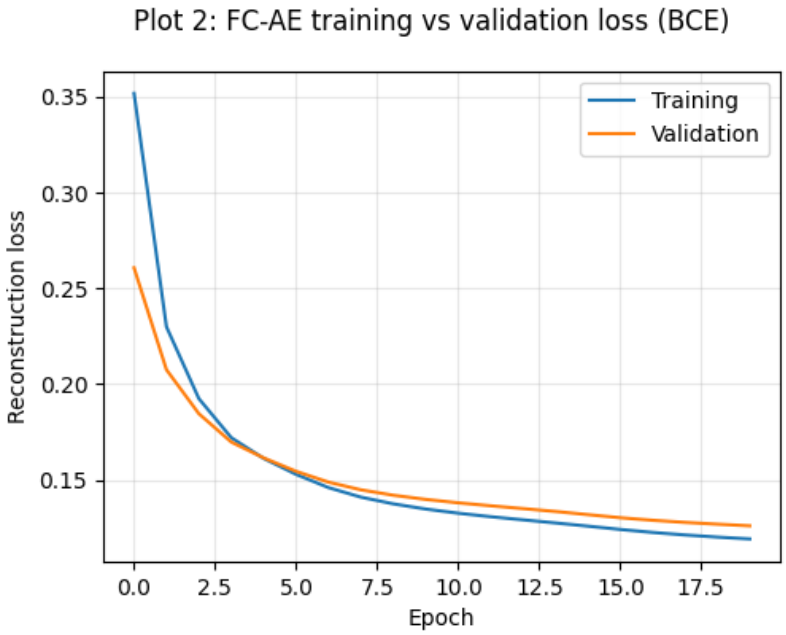


Plot 1: Original images, FC-AE reconstructions and absolute error for ten test images.

Inference: The digit shapes survive reconstruction. Strokes look smoothed. Fine details are lost. This is clearest for thin or curved digits like the 9 and 5. The error maps sit along stroke edges. Harder samples show larger errors.

Training and Validation Reconstruction Loss

- X-axis: Epoch
- Y-axis: Reconstruction loss
- Curves: Training loss and validation loss



Plot 2: Training and validation binary cross-entropy of the FC-AE (latent dimension 16); epoch 0 on the axis is the first epoch.

Inference: Training BCE drops from 0.35 to 0.12. Validation BCE drops from 0.26 to 0.13. Most of the drop happens in the first five epochs. Both curves are still falling slowly. The small gap suggests slight overfitting.

9. Reconstruction Metrics

Students must calculate the following metrics on the test set.

Mean Squared Error

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|x_i - \hat{x}_i|.$$

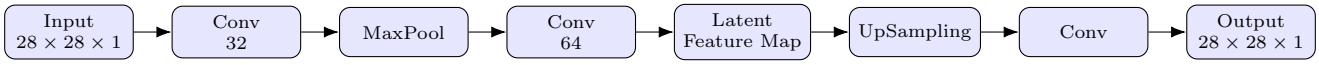
Structural Similarity

Calculate SSIM between the original and reconstructed images.
The implementation may use:
`https://scikit-image.org/docs/stable/api/skimage.metrics.html`
Report:

Metric	Value
Test MSE	0.020457
Test MAE	0.055120
Mean SSIM	0.758654

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.
The conceptual architecture is:



A possible implementation is:

```
Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

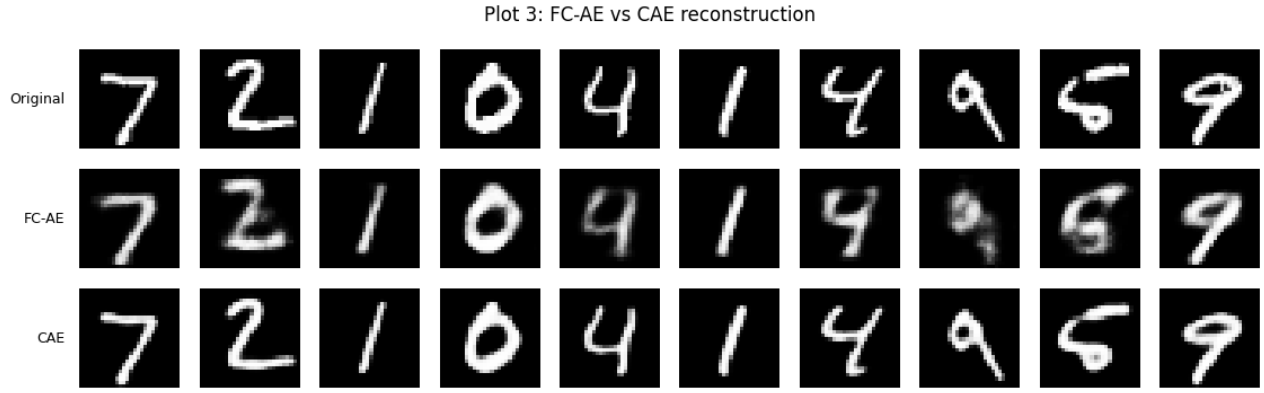
11. Comparing Fully Connected and Convolutional Autoencoders

Train both models using the same training and test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.020457	0.055120	0.758654	211040
Convolutional AE	0.002645	0.015040	0.974012	74497

Reconstruction Comparison
Display:

Original → FC-AE Reconstruction → CAE Reconstruction.



Plot 3: Original images, FC-AE reconstructions and CAE reconstructions.

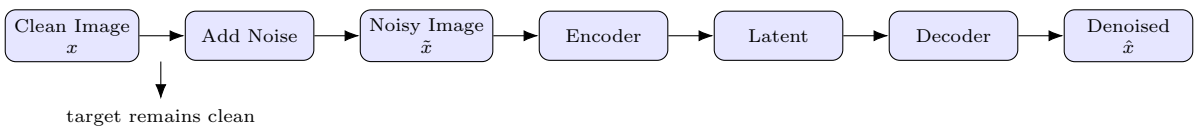
Inference: The CAE has lower MSE and higher SSIM. It keeps digit contours sharp. The FC-AE blurs them. The CAE latent map has 7*7*64=3136 values. The FC-AE code has only 16. So part of the gain comes from extra capacity.

12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image.
The training mapping is:

$$\boxed{\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}}$$

where $\tilde{x}$ is a noisy version of the clean image x.



13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \qquad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

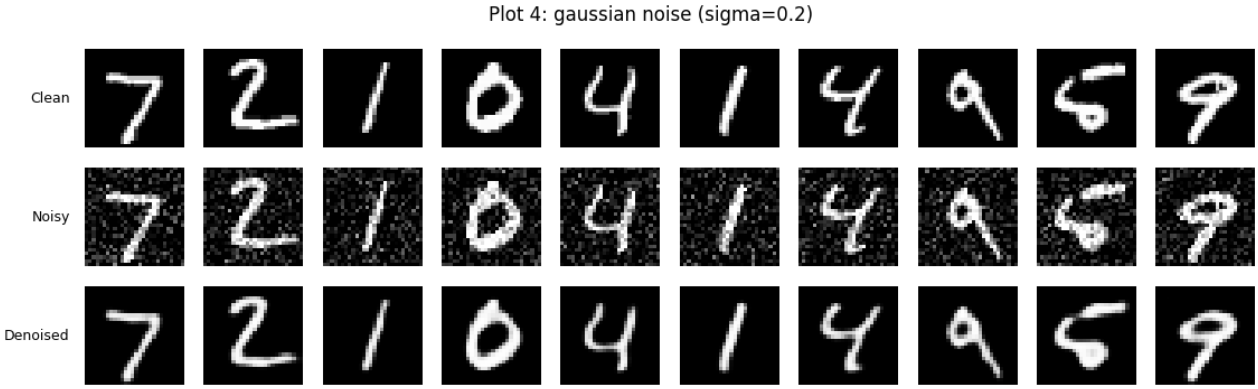
Randomly replace a small proportion of pixels with values close to 0 or 1.
Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

14. Required Denoising Plots

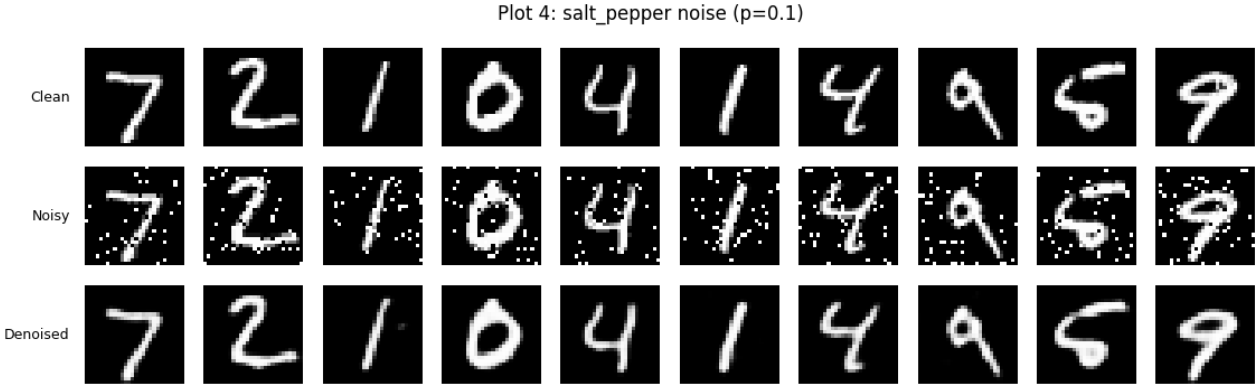
Clean vs. Noisy vs. Denoised
For at least 10 images, display:

Clean | Noisy | Denoised.



Plot 4: Gaussian noise. Clean, noisy ($\sigma = 0.2$) and denoised images.

Inference: At $\sigma = 0.2$ the noisy inputs look heavily dotted. The denoised outputs are clean digits. The background is near black. The main structure as desired is regenerated decently.



Plot 5: Salt-and-pepper noise. Clean, noisy ($p = 0.10$) and denoised images.

Inference: Isolated black and white pixels are removed. The digits keep their shape. The thin 1 (third image) is the weakest case.

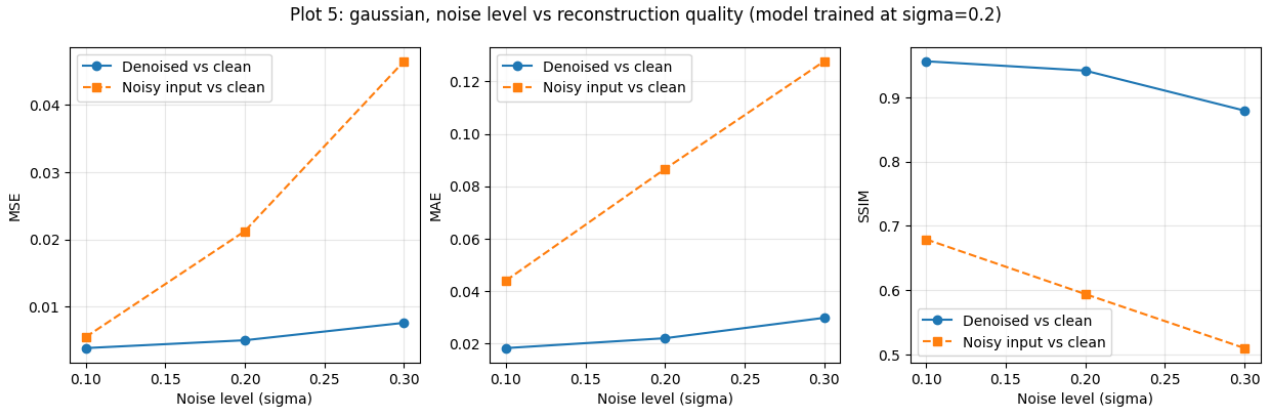
Noise Level vs. Reconstruction Quality

For each noise level, calculate:

$$MSE, \quad MAE, \quad SSIM.$$

Report:

Noise Level	MSE	MAE	SSIM
0.1	0.003786	0.018285	0.956065
0.2	0.004954	0.022048	0.941529
0.3	0.007552	0.029823	0.879364



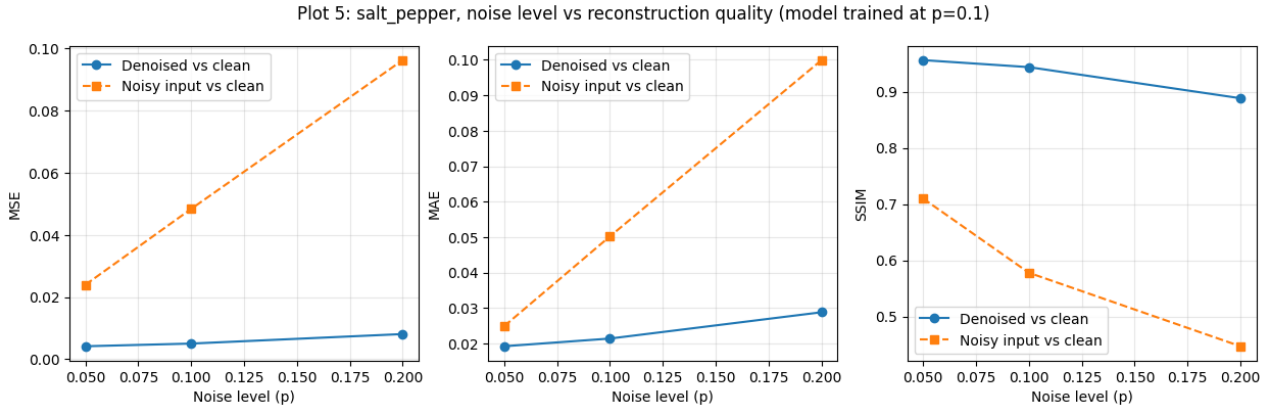
Plot 6: Gaussian noise. MSE, MAE and SSIM versus σ for the denoised output and the noisy input (model trained at $\sigma = 0.2$).

Inference: As σ rises from 0.1 to 0.3, the noisy input MSE grows quite a bit. The denoised MSE doubles, from 3.8×10^{-3} to 7.6×10^{-3}. SSIM falls from 0.96 to 0.88. Most of the noise is removed. Quality still drops as noise increases.

Additional salt-and-pepper evaluation:

Noise Level	MSE	MAE	SSIM
0.05	0.004171	0.019201	0.956170
0.10	0.005031	0.021376	0.943708
0.20	0.008098	0.028795	0.888340

The salt and pepper results show the same trend. Stronger corruption raises error and lowers SSIM. The denoiser still keeps the main digit structure at every level.



Plot 7: Salt-and-pepper noise. MSE, MAE and SSIM versus p for the denoised output and the noisy input (model trained at $p = 0.10$).

Inference: From $p = 0.05$ to 0.20, the noisy-input MSE grows from 0.02 to 0.10. The denoised MSE stays between 4.2×10^{-3} and 8.1×10^{-3}. SSIM falls only from 0.96 to 0.89.

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

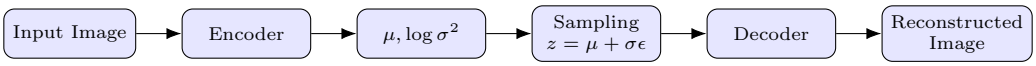
$$q_\phi(z|x) = \mathcal{N}\left(\mu(x), \mathrm{diag}(\sigma^2(x))\right).$$

The encoder produces:

$$\mu, \qquad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$



16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}\left(q_\phi(z|x) \parallel p(z)\right) = -\frac{1}{2}\sum_j \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

17. VAE Implementation

Use a small latent space so that it can be visualized.
Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

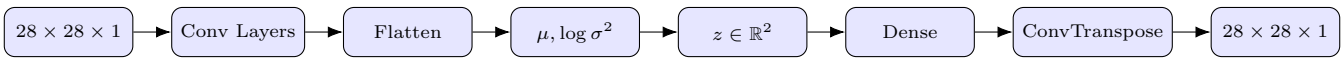
The encoder should produce:

$$\mu \in \mathbb{R}^2, \qquad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \to 28 \times 28 \times 1.$$

A compact architecture is:



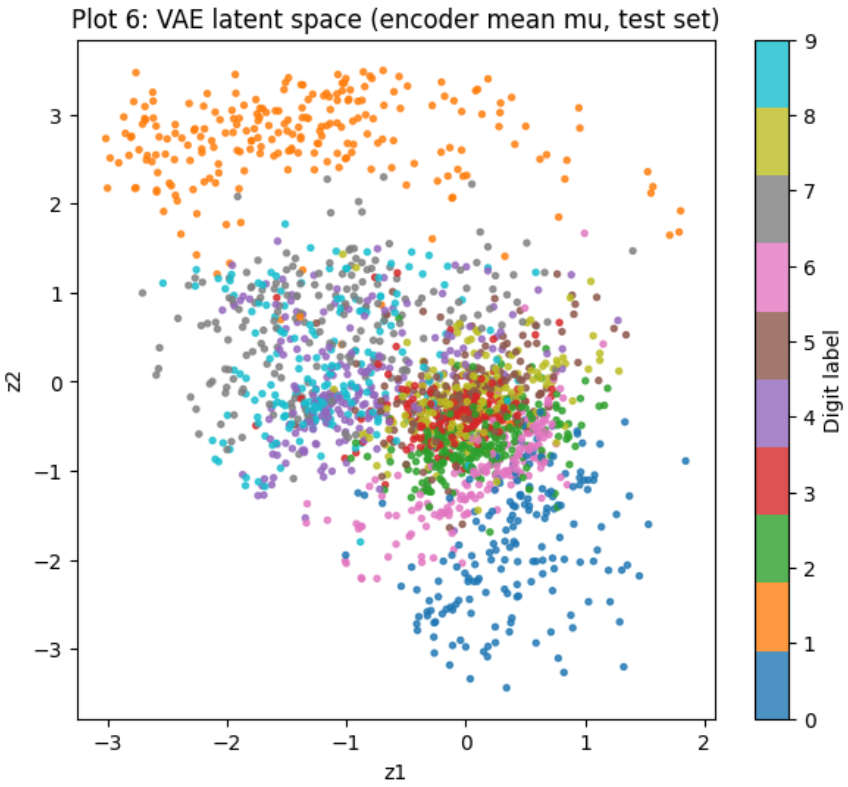
18. Exploring the VAE Latent Space

Since the latent dimension is two, students can visualize the encoded test images in a two-dimensional plane.

VAE Latent Space

- X-axis: z_1
- Y-axis: z_2
- Color/marker category: digit label

The labels are used only for visualization; they are not supplied to the VAE during training.



Plot 8: Two-dimensional VAE latent space (encoder mean μ of the test images, coloured by digit label).

Inference: Digit 1 (top) and digit 0 (lower right) form separate regions. Most other digits overlap heavily. The 4, 7 and 9 mix on the left. The 2, 3, 6 and 8 sit near the origin.

19. Generating New Images with the VAE

Sample points from

$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

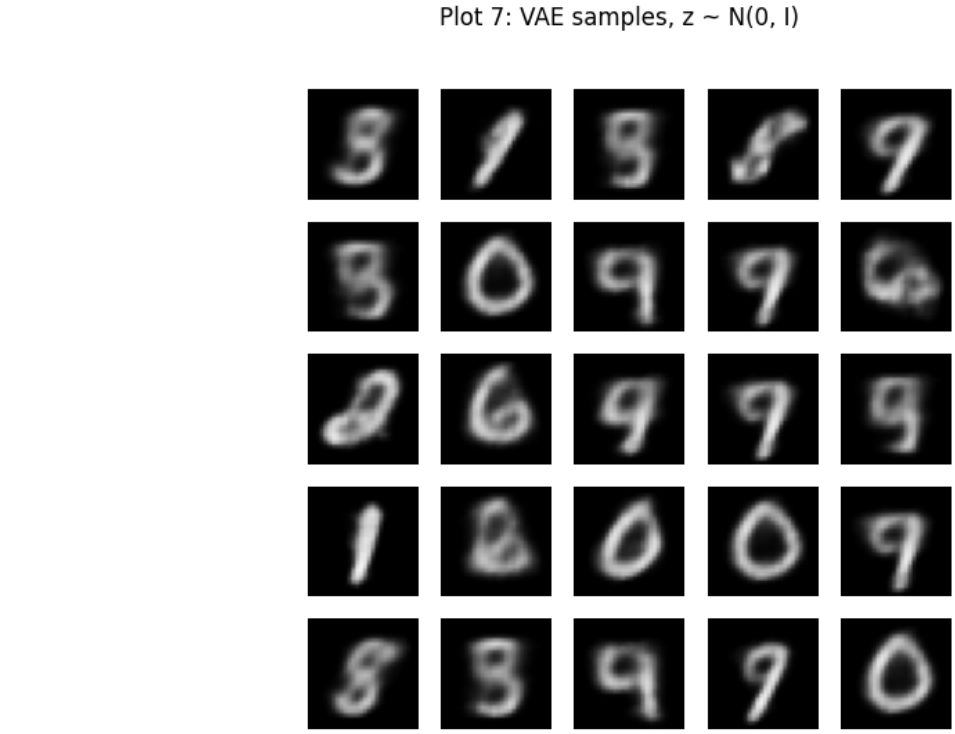
Randomly Generated Images

Generate at least 25 images using randomly sampled latent vectors.

Display them in a grid.

Students should determine:

- whether the generated images resemble handwritten digits;
- whether digit shapes are recognizable;
- whether there is visible variation among generated samples; and
- whether some samples are ambiguous or unrealistic.



Plot 9: 25 images generated by decoding .

Inference: Most samples are blurry but decently resemble digits. The 0, 1, 6 and 9 shapes are the clearest. There is visible variation between samples. Several are ambiguous blobs.

20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \qquad z_B.$$

Construct

$$z(\alpha) = (1-\alpha)z_A + \alpha z_B, \qquad \alpha \in [0,1].$$

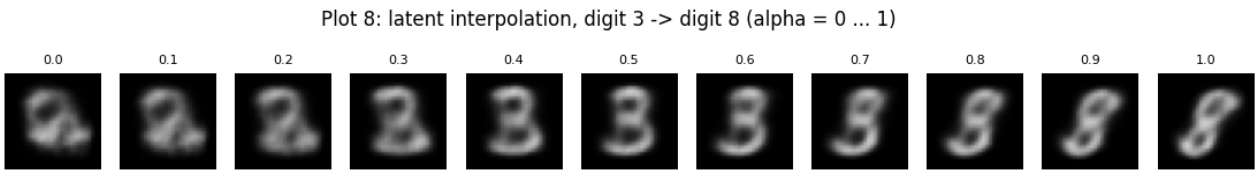
Use values such as:

$$\alpha = 0,\ 0.1,\ 0.2, \ldots, 1.$$

Decode every interpolated vector.

Latent Space Interpolation

Display the decoded images in sequence.



Plot 10: Decoded images along the latent path from a test "3" to a test "8" ($\alpha = 0, 0.1, \ldots, 1$).

Inference: The image starts as a blurry 3 like shape. It turns into a clear 8 like shape by $\alpha \approx 0.6$. There are no sudden jumps.

21. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

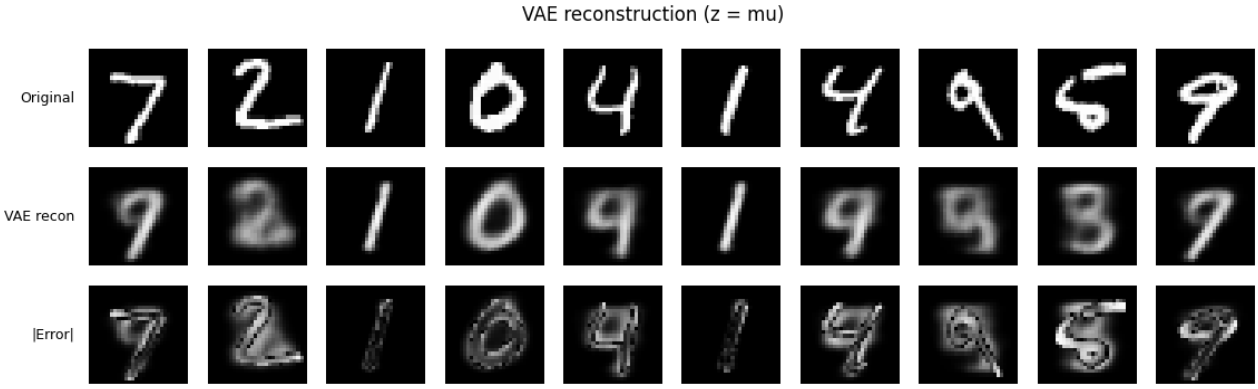
Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.020457	0.055120	0.758654	211040
Convolutional AE	0.002645	0.015040	0.974012	74497
Denoising CAE (Gaussian, $\sigma = 0.2$)	0.004936	0.022006	0.941866	74497
Denoising CAE (salt&pepper, $p = 0.10$)	0.005066	0.021445	0.943521	74497

The denoising rows are evaluated on noisy test inputs against the clean images, so they are not directly comparable with the clean-input rows. .

For the VAE, report its reconstruction loss separately because its total objective also contains the KL-divergence term.

VAE Metric	Value
Reconstruction Loss	152.510406
KL Loss	5.683816
Total Loss	158.194223
Test MSE	0.045047
Test MAE	0.104251
Mean SSIM	0.472787

The VAE losses are summed over the 784 pixels (or latent dimensions) of each image and averaged over the test set.



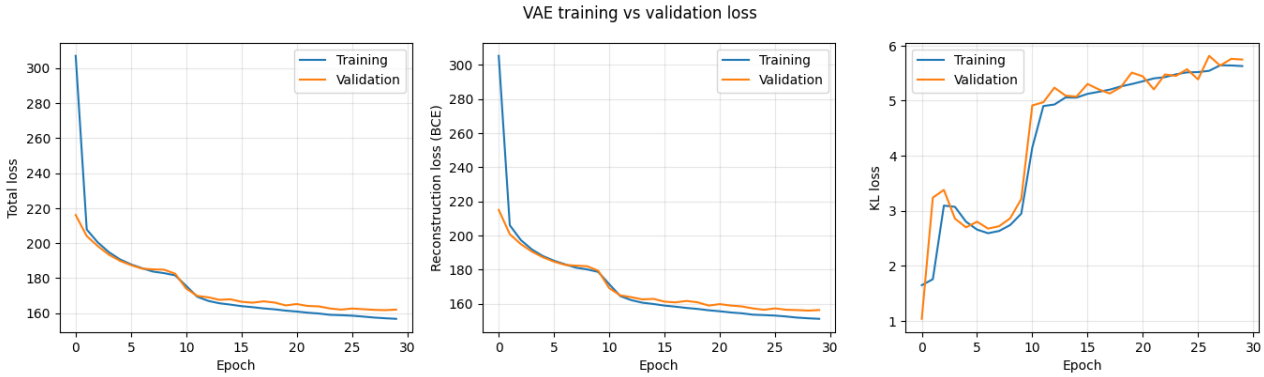
Plot 11: VAE reconstructions using $z = \mu$. Original images, reconstructions and absolute error.

Inference: The reconstructions keep only the coarse digit shape. They are blurrier than the AE and CAE outputs. Some digits change identity. The 7 and 4 drift toward 9. The 5 drifts toward 3. A two-dimensional code cannot store enough detail. MSE is 0.05 and SSIM is 0.47.

22. Required Plots

The final report must contain the following plots:

1. Original image versus reconstructed image.
2. Training and validation loss for the basic autoencoder.
3. Fully connected AE versus convolutional AE reconstruction.
4. Clean versus noisy versus denoised images.
5. Noise level versus MSE/MAE/SSIM.
6. VAE latent-space visualization.
7. Randomly generated VAE images.
8. Latent-space interpolation.
9. Training/validation reconstruction loss for the VAE.
10. Reconstruction-error distribution on the test set.



Plot 12: VAE training and validation loss. Total, reconstruction (BCE) and KL terms; epoch 0 on the axis is the first epoch.

Inference: Training and validation losses stay close. They end at 156.70 and 161.96. Overfitting is mild. Around epochs 10–12 the KL term jumps from 2.9 to 4.9. The reconstruction loss drops at the same time. The encoder seems to use the latent code more.

For plots containing multiple quantitative measures, use separate figures or clearly labeled axes rather than placing incompatible scales on one axis.

23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

$$e_i = \frac{1}{D}\sum_{j=1}^{D}(x_{ij} - \hat{x}_{ij})^2.$$

Distribution of Reconstruction Errors

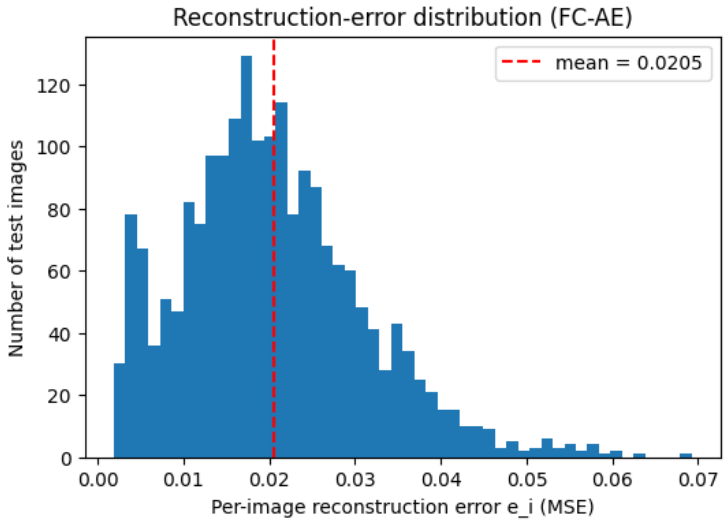
Plot a histogram of e_i over the test set.

Students should identify:

- the typical reconstruction-error range;
- whether the distribution is concentrated or spread out;
- possible high-error samples; and
- representative images corresponding to high reconstruction error.

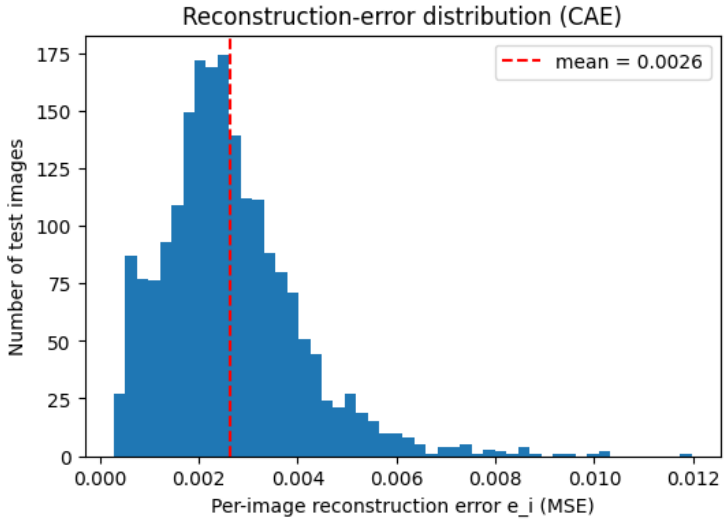
FC-AE distribution: mean = 0.02046, median = 0.01937, standard deviation = 0.01064, minimum = 0.00185, maximum = 0.06921, and 95th percentile = 0.03934.

CAE distribution: mean = 0.00265, median = 0.00244, standard deviation = 0.00141, minimum = 0.00028, maximum = 0.01196, and 95th percentile = 0.00516.



Plot 13: Distribution of per-image reconstruction error on the test set (FC-AE).

Inference: The FC-AE errors are right-skewed and spread out. The mean is 0.02. The median is 0.02. The standard deviation is 0.01. A tail of hard images reaches 0.07. Most images lie below the 95th percentile of 0.04.



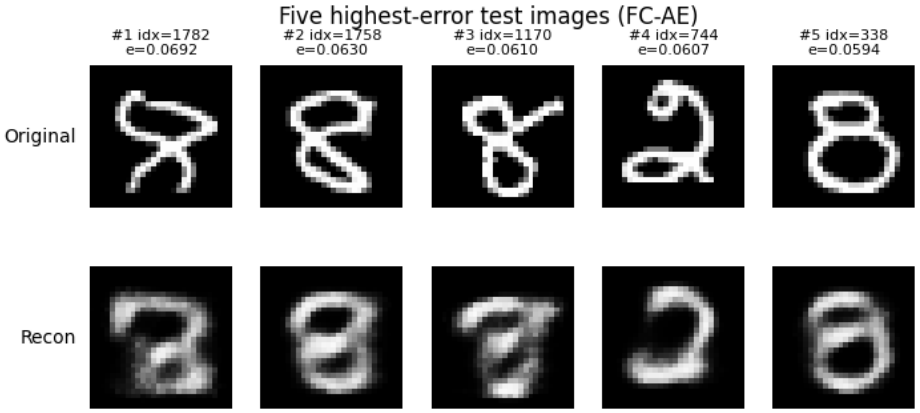
Plot 14: Distribution of per-image reconstruction error on the test set (CAE).

Inference: The CAE errors are concentrated at a much lower level. The mean is 2.7×10^{-3}. The 95th percentile is 5.2×10^{-3}. The right tail reaches 0.01. Even the worst image beats the FC-AE mean. This matches the gap seen in Plot 3.

24. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.
Display:
FC-AE highest-error samples:

Sample	Test index	Reconstruction error	Observation
1	1782	0.0692	Crossed/looped strokes
2	1758	0.0630	Large looped 8
3	1170	0.0610	8 with an extra tail stroke
4	744	0.0607	2 with a loop
5	338	0.0594	Thick looped 8

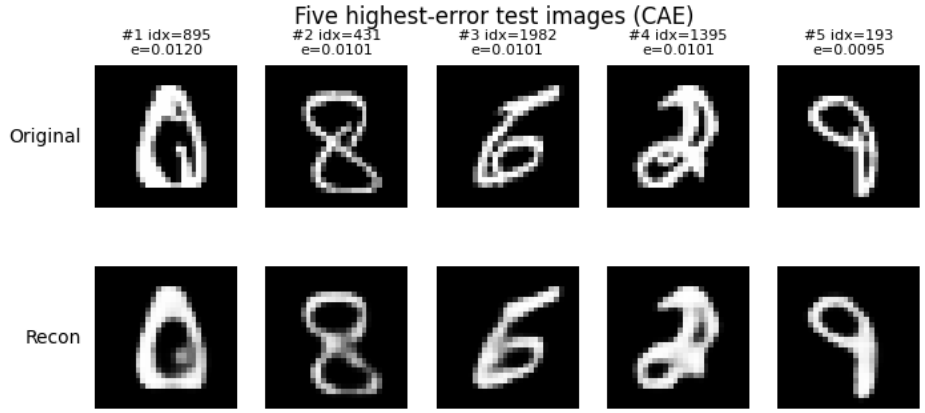


Plot 15: Five test images with the highest reconstruction error (FC-AE). Originals and reconstructions.

Inference: The five worst FC-AE errors range from 0.06 to 0.07. Most are looped 8-like shapes with crossing strokes. One is a curly 2. The 16-dimensional code cannot capture these strokes. Outputs turn blurry. Samples 1 and 3 change shape.

CAE highest-error samples:

Sample	Test index	Reconstruction error	Observation
1	895	0.0120	0 with an inner mark
2	431	0.0101	Thin looped 8
3	1982	0.0101	6 with a long stroke
4	1395	0.0101	Thick looped 2
5	193	0.0095	9 with extra loops



Plot 16: Five test images with the highest reconstruction error (CAE). Originals and reconstructions.

Inference: The five worst CAE errors are all about 0.01. These are unusual writing styles. One is a 0 with an inner mark. The rest are thin or looped 8, 6, 2 and 9 shapes. They stay recognizable. Fine detail is lost. The inner mark of the 0 becomes a grey blob.

25. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time (s)
FC Autoencoder	0.020457	0.055120	0.758654	211040	14.856031
Conv. Autoencoder	0.002645	0.015040	0.974012	74497	19.337252
Denoising CAE (Gaussian, $\sigma = 0.2$)	0.004936	0.022006	0.941866	74497	16.619962
Denoising CAE (salt&pepper, $p = 0.1$)	0.005066	0.021445	0.943521	74497	16.975360
VAE ($z = \mu$)	0.045047	0.104251	0.472787	134165	28.331639

Time is the wall-clock training time; the first epoch alone took about 6–14 s in these runs (graph compilation and warm-up), so it should not be read as a ranking of model cost. The VAE parameter count covers encoder and decoder.

The values must be obtained from the student's own execution.

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn?

The following inferences are mandatory:

- **Latent dimension:** Raising the FC-AE latent dimension from 2 to 32 cut MSE from 0.05 to 0.02. SSIM rose from 0.45 to 0.78. More capacity improves reconstruction. The cost is weaker compression and slightly more parameters.
- **FC versus convolutional reconstruction:** The CAE reached MSE 2.65×10^{-3} and SSIM 0.97. The FC-AE reached 0.02 and 0.76. Convolutions preserve spatial locality. So the CAE reproduces contours better. Its larger latent map ($7 \times 7 \times 64$) also helps.
- **Gaussian noise:** For the denoising CAE, raising σ from 0.1 to 0.3 increased MSE from 3.79×10^{-3} to 7.55×10^{-3}. SSIM fell from 0.96 to 0.88. Stronger noise makes reconstruction harder.
- **Denoising ability:** At $\sigma = 0.2$ the noisy input had MSE 0.02 and SSIM 0.59. The denoised output had MSE 4.95×10^{-3} and SSIM 0.94. The model removes much of the noise. It keeps the major digit structure.

- **VAE latent distribution:** The VAE encodes each input as μ and $\log \sigma^2$. It does not use a single fixed code. The two-dimensional codes show some structure. Several digit classes still overlap.
- **Two-dimensional latent space:** Some digit groups sit in recognizable regions. Several classes overlap near the center. The coordinates capture meaningful variation. They do not separate all digits.
- **Generated samples:** Random samples from $\mathcal{N}(0, I)$ are mostly blurry but digit-like. They show visible variation. Several are ambiguous or unreadable. Quality is uneven across the prior space.
- **Latent interpolation:** The 3 to 8 interpolation changes gradually. Intermediate images form transitional shapes. There is no abrupt switch between the endpoints.
- **Reconstruction error and visual quality:** The FC-AE has a mean per-image error of 0.02. Its worst cases show poor reconstruction of complex strokes. The CAE has a lower mean error of 2.65×10^{-3}. Its reconstructions look closer to the originals.
- **Deterministic versus probabilistic representations:** The FC-AE maps an image to one fixed latent vector. The VAE learns a distribution with μ and σ. This enables sampling and generation. It also adds a KL term. The VAE MSE (0.05) is close to the 2-dimensional FC-AE (0.05). Both are higher than the 16-dimensional FC-AE (0.02). So the gap mostly reflects the small latent size.

27. Additional Latent-Dimension Study

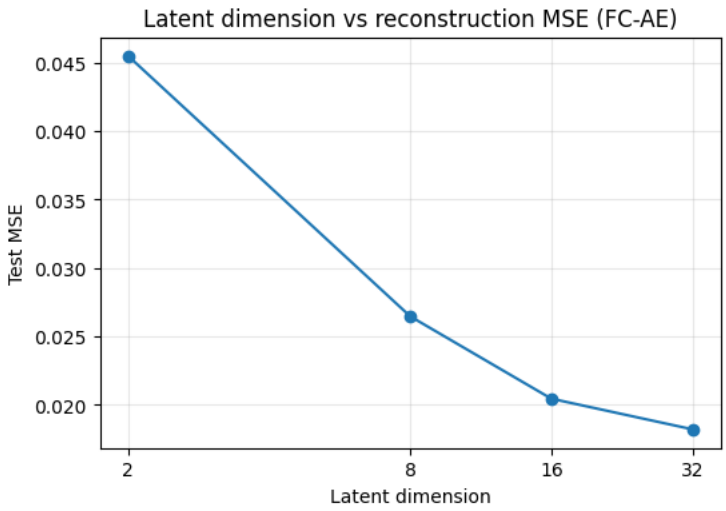
Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

Report:

Latent Dimension	MSE	SSIM	Parameters
2	0.045486	0.454099	210130
8	0.026468	0.688646	210520
16	0.020457	0.758654	211040
32	0.018221	0.779561	212080

Latent Dimension vs. Reconstruction MSE



Plot 17: Test MSE of the FC-AE for latent dimensions 2, 8, 16 and 32.

Inference: Test MSE falls from 0.05 at dimension 2 to 0.02 at dimension 32. SSIM rises from 0.45 to 0.78. The gains shrink. From 16 to 32, MSE drops only about 11%. Larger latent spaces keep more information. They also compress less. The trade-off flattens beyond 16 dimensions.

28. Discussion Questions

1. **What is an autoencoder?**
   An autoencoder is a neural network trained to reconstruct its input as well as possible. It strives to minimise reconstruction error.

2. **What is the purpose of the encoder?**
   The encoder maps the input image to a lower dimensional latent representation. It extracts useful information to reconstruct.
3. **What is the purpose of the decoder?**
   The decoder maps the latent representation back to the original image space. Its output is the reconstructed image.
4. **What is a latent representation?**
   It is the compact representation produced by the encoder. It contains the features retained from the input for reconstruction. It retains the features that are most important.
5. **Why is the latent representation usually smaller than the input?**
   A smaller latent space forces the model to compress the important information. Useful features must be extracted, not everything.
6. **What happens when the latent dimension is too small?**
   Very less latent capacity causes information loss and poorer reconstructions.
7. **What happens when the latent dimension is increased?**
   More information can be retained, generally improving reconstruction quality. However may lead to overfitting.
8. **Why can a convolutional autoencoder preserve spatial information more effectively?**
   Convolutions operate on local neighborhoods and retain spatial relationships between pixels. This is better for images since spatial relationships exist. Pixels are not independent.
9. **Differentiate an autoencoder and a denoising autoencoder.**
   A conventional autoencoder reconstructs the same clean input it receives. A denoising autoencoder receives a noisy image; it reconstructs the clean one.
10. **Why is the clean image used as the target in denoising?**
    The clean image provides the desired output without noise. This trains the network to remove it.
11. **What happens when the noise level is increased?**
    Reconstruction becomes more difficult, as the baseline for what a good image is changed. The image characteristics may itself end up changing.
12. **Why are MSE, MAE and SSIM useful for image reconstruction?**
    MSE measures squared pixel error and penalizes larger errors strongly, while MAE measures average absolute error. SSIM focuses on structural similarity between images.
13. **What is a Variational Autoencoder?**
    A VAE is an autoencoder that learns a probability distribution in latent space. It uses μ and σ to sample latent vectors and can therefore generate new samples.
14. **How does a VAE differ from a conventional autoencoder?**
    A conventional autoencoder maps an input to a deterministic latent vector. A VAE learns a latent distribution and adds KL divergence.
15. **Why does a VAE learn a distribution instead of a single latent vector?**
    A distribution makes the latent space continuous and easier to sample. Newer latent spaces can be reconstructed.
16. **What is the reparameterization trick?**
    This method separates random sampling from the learnable parameters so gradients can pass through the computation.
17. **Why is ϵ sampled from a standard normal distribution?**
    The standard normal provides a fixed source of randomness with zero mean and unit variance. The encoder then controls the resulting distribution.
18. **What is the role of KL divergence in the VAE loss?**
    KL divergence regularizes the learned latent distribution making it more structured and suitable for sampling.
19. **What is the prior distribution used in the VAE?**
    The experiment uses the standard normal prior $p(z) = \mathcal{N}(0, I)$. Random generation samples latent vectors from this distribution.
20. **Why is a two-dimensional latent space useful for visualization?**
    Two dimensions can be plotted directly as a coordinate plane. This makes clusters and transitions easy to represent and interpret.
21. **What does latent-space interpolation demonstrate?**
    Interpolation tests whether nearby latent points decode to smoothly changing images. The experiment demonstrates how the digit images gradually evolve.
22. **Why can a VAE generate new images?**
    The VAE learns a continuous probabilistic latent space and a decoder from latent vectors to images. If a latent vector is being sampled from the prior and decoding it therefore produces a new image.
23. **Why might some VAE-generated images be unclear?**
    The decoder may receive latent points that are not strongly associated with a clear training example. This makes it less clear.
24. **What is the difference between reconstruction and generation?**

Reconstruction encodes an existing image and decodes its latent representation back to an approximation of that image. Generation starts from a sampled latent vector and produces a new image and is unsupervised.

25. **Why should test images remain unseen during training?**
Unseen test images provide an unbiased measure of how well the model generalizes. Using them during training defeats the purpose.

29. Additional Exercises

1. Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.
Observation: Test MSE falls from 0.03 to 5.97×10^{-3} as the dimension grows from 4 to 32. SSIM rises from 0.64 to 0.94. A larger code keeps more detail. It also compresses less. Even at 32, this CAE trails the main CAE (MSE 2.65×10^{-3}, SSIM 0.97). That model has a $7 \times 7 \times 64$ latent map. Capacity clearly matters.
2. Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.
Observation: Each model does best on its own noise type (0.94 and 0.95). The Gaussian-trained model transfers fairly well to salt-and-pepper noise (0.88). The reverse transfer is poor (0.63). A likely reason is that removing isolated impulse pixels does not teach smoothing of diffuse noise. The Gaussian-trained model learns the more general behaviour.
3. Train the denoising autoencoder using two different noise levels and compare SSIM.
Observation: The $\sigma = 0.1$ model is best on light noise (0.96). It degrades fast and falls to 0.7 at $\sigma = 0.3$. The $\sigma = 0.3$ model is slightly worse on light noise (0.95). It is far more robust and keeps 0.92 at $\sigma = 0.3$. Training on stronger noise gives more consistent results. Training on weak noise specialises the model.
4. Replace the convolutional decoder with transposed convolutions and compare the reconstructions.
Observation: Both decoders have 74497 parameters. The transposed-convolution decoder has lower MSE (2.17×10^{-3} vs 2.80×10^{-3}). It also has higher SSIM (0.98 vs 0.97). Its upsampling is learned, not fixed. The gain is small but consistent.
5. Implement a VAE with latent dimension 2 and 8 and compare the latent representations.
Observation: Going from 2 to 8 dimensions lowers reconstruction loss from 154.6 to 140.4. SSIM rises from 0.47 to 0.56. The KL term rises from 6.58 to 9.13. More information is being encoded. All dimensions stay active, so there is no posterior collapse. The 8-dimensional model reconstructs better. It cannot be plotted directly. The 2-dimensional values differ slightly from Section 21. This likely reflects a separate training run.
6. Investigate the effect of increasing the KL-loss contribution.
Observation: As β rises from 0.5 to 10, KL falls from 7.18 to 1.94. Reconstruction loss rises from 150.2 to 168.3. SSIM drops from 0.49 to 0.39. A stronger KL weight pulls the posterior toward the prior. The latent space becomes more regular. Each code carries less information. So reconstructions worsen. This is the trade-off between fidelity and regularisation.
7. Generate a larger grid of VAE samples and discuss diversity.
Observation: Generated images are less diverse than real digits (6.93 and 5.63 against 9.84). The VAE covers less variation than the data. The 8-dimensional VAE reconstructs better. Its samples are less diverse than the 2-dimensional ones. A plausible reason is that prior samples decode to blurrier, more average digits. That lowers pairwise distances.
8. Select two digit regions in the latent space and perform interpolation.
Observation: The $1 \to 0$ path shows a clear, gradual transition. The largest step (0.03) is under three times the mean (9.87×10^{-3}). There is no single abrupt jump. The $4 \to 9$ path shows almost no change (4.00×10^{-5}). The endpoints decode to nearly the same image. This fits Plot 8, where 4 and 9 overlap heavily in the latent space.

30. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

$$\boxed{\text{Image} \to \text{Encoder} \to \text{Latent Representation} \to \text{Decoder} \to \text{Reconstructed Image}}$$

They should also demonstrate denoising:

$$\boxed{\text{Clean Image} \to \text{Noise} \to \text{Denoising Autoencoder} \to \text{Recovered Image}}$$

and variational generation:

$$\boxed{\text{Image} \to (\mu, \sigma) \to z \to \text{Decoder} \to \text{Generated/Reconstructed Image}}$$

The final report should contain both quantitative evaluation and visual analysis. Numerical performance values must be obtained from the student's own execution.

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.
3. Pascal Vincent et al., "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, "MNIST Handwritten Digit Database."
5. TensorFlow Documentation: `https://www.tensorflow.org`
6. Keras Documentation: `https://keras.io`
7. scikit-image Documentation: `https://scikit-image.org`